

Yes. Mas maganda ngayon na **hindi na generic yung master prompt**. I-embed na natin mismo sa Claude Code ang concept ng app para alam niya kung *ano* ang ginagawa niya, habang yung reference image ang magiging product-development framework.

Ang app concept mo, in simple terms:

A personal Philippine stock portfolio tracker — authenticated users can maintain their own PSE stock portfolio, track holdings and portfolio value/growth, and read Philippine stock-market news.

Ito ang mas complete na prompt na puwede mong i-paste sa Claude Code:

Edit

CLAUDE CODE — PHILIPPINE STOCK PORTFOLIO APP

You are the lead product engineer for this application.

You are working inside an existing project that is already configured with:

- Next.js
- Expo
- React Native
- TypeScript
- Existing project dependencies and configuration

Do not replace the existing stack unless there is a strong technical reason.

1. APPLICATION CONCEPT

We are building a simple personal finance application focused specifically on **Philippine stocks / PSE-listed stocks**.

The application is primarily a **personal Philippine stock portfolio tracker**.

The user should be able to:

1. Create an account and authenticate.
2. See a list of Philippine stocks.
3. Create and maintain their own portfolio.
4. Add stock holdings.
5. Increase or decrease the quantity of a holding.
6. See the current value of individual holdings.
7. See the total value of their portfolio.
8. See the growth/performance of their portfolio through a graph.
9. Read Philippine stock-market news.
10. Manage basic application settings.

Each authenticated user must have their own isolated portfolio data.

A user must NEVER be able to see another user's portfolio, holdings, account information, or private data.

2. PRODUCT PRINCIPLE

The application should remain intentionally simple.

Use this product structure as the primary product-development framework:

RAW IDEA



CORE FUNCTION



CORE LOOP



ACCESSORY FEATURES



SURFACE AREA CHECK



RETENTION HOOK



SHIPPABLE MVP

The provided product-design reference image represents this philosophy.

Do not turn this application into a complicated trading terminal.

This is NOT initially intended to be:

- a brokerage platform
- a stock trading execution platform
- a complicated financial dashboard
- a professional trading terminal
- a social network
- a cryptocurrency application
- a multi-country stock platform

The focus is:

"Let me quickly understand what I own, how much my Philippine stock portfolio is worth, how it is performing, and what is happening in the Philippine stock market."

3. CORE FUNCTION

The ONE core function is:

Allow a user to track their Philippine stock portfolio and quickly understand its current value and performance.

Everything else should support this.

If a feature does not support that purpose, do not automatically add it.

4. CORE LOOP

The main product loop is:

OPEN APP

↓

SEE PORTFOLIO VALUE

↓

SEE HOLDINGS / PERFORMANCE

↓

ADD OR UPDATE A HOLDING

↓

PORTFOLIO VALUE UPDATES



SEE PERFORMANCE / GROWTH



RETURN LATER TO CHECK PORTFOLIO

The loop should be understandable almost immediately.

A user should not need to navigate through multiple complicated screens just to understand their portfolio.

5. PRIMARY APPLICATION SURFACES

The MVP should have a small number of primary surfaces.

Target approximately 4–5 primary tabs/screens.

A suggested structure is:

1. Portfolio / Home

The primary screen.

Show:

- Total portfolio value
- Portfolio growth/performance
- Portfolio performance graph
- Holdings list
- Individual stock
- Quantity held
- Current value
- Gain/loss where data permits
- Primary action to add/update holdings

This should be the most important screen in the application.

2. Stocks

A searchable/listed collection of Philippine stocks.

Show information such as:

- Stock symbol
- Company name
- Current price
- Basic price information
- Relevant change/performance information when available

The initial scope is **Philippine stocks only**.

Do not add US stocks or other international markets unless explicitly requested later.

3. News

A Philippine stock-market news feed.

The news section should aggregate relevant Philippine stock-market/business news from appropriate finance/business news sources.

Potential sources may include reputable Philippine financial/business publications and other legitimate sources that provide relevant Philippine stock-market information.

The news experience should include:

- headline
- source
- publication date/time when available
- image when available
- short summary/excerpt when legally and technically appropriate
- link to the original article

Do not scrape or reproduce entire copyrighted articles.

The app should link users to the original source.

News should primarily focus on:

- Philippine stocks
- PSE
- Philippine companies
- Philippine markets

- corporate announcements
 - earnings
 - market movements
 - relevant economic developments affecting Philippine equities
-

4. Settings

Basic settings such as:

- Account information
- Profile
- Authentication/account management
- Preferences
- Logout
- Other simple app preferences

Do not turn Settings into a giant configuration screen.

6. AUTHENTICATION AND DATA ISOLATION

Authentication is a core requirement.

Users must be able to:

- Sign up
- Log in
- Log out
- Maintain an authenticated session

Each user's portfolio must be private to that authenticated account.

Conceptually:

USER A

→ Account A

→ Portfolio A

→ Holdings A

USER B

→ Account B

→ Portfolio B

→ Holdings B

User A must never receive User B's private portfolio data.

This must be enforced at the data/backend/security layer, not merely by hiding UI elements.

Do NOT rely on frontend filtering as the security mechanism.

When we implement the backend/database:

- every private portfolio record must belong to a user/account
- authenticated requests must be associated with the correct user
- database access must enforce user isolation
- users must only be able to read/write their own portfolio and holdings

Before implementing the final authentication/database architecture, inspect the existing project and use the existing infrastructure if one already exists.

7. PORTFOLIO MODEL

The portfolio should allow a user to create holdings for Philippine stocks.

A holding should conceptually contain information such as:

- Stock symbol
- Company
- Quantity
- Average purchase price, if tracked
- Current price
- Current value
- Gain/loss, if enough data exists
- User ownership
- Relevant timestamps

The user should be able to:

ADD

Add a new stock holding.

Example:

BDO
100 shares

INCREASE

Increase the quantity of an existing holding.

Example:

BDO
100 → 150 shares

DECREASE

Decrease the quantity.

Example:

BDO
150 → 100 shares

REMOVE

If quantity reaches zero, allow the holding to be removed or archived appropriately.

The UI should make these actions extremely simple.

8. PORTFOLIO VALUE

The main portfolio screen should clearly display:

TOTAL PORTFOLIO VALUE

This is the most important number.

The user should also be able to see the contribution of individual holdings.

Example conceptual layout:

TOTAL PORTFOLIO

₱125,450.00

+₱8,250

+7.04%

Then:

BDO

100 shares

₱18,500

SM

50 shares

₱42,000

ALI

100 shares

₱28,000

etc.

The exact values should come from the application's data layer once real market data is connected.

During Phase 1, mock data is acceptable.

9. PORTFOLIO PERFORMANCE GRAPH

The application must have a portfolio performance graph.

The graph should help answer:

"How has my portfolio changed over time?"

The UI can eventually support time ranges such as:

- 1D
- 1W
- 1M
- 3M

- 6M
- 1Y
- All

However, do not automatically implement every range in Phase 1.

Start with the simplest useful representation.

The graph should be visually clean and easy to understand.

Avoid turning it into a professional trading chart with dozens of indicators.

The graph is primarily for **portfolio growth/performance**, not advanced technical analysis.

10. PHILIPPINE STOCK DATA

The application is specifically for Philippine stocks.

Use PSE-listed stocks as the primary universe.

The architecture should make it possible to retrieve/update:

- stock symbols
- company names
- current prices
- price changes
- historical prices where available
- other basic market data required for portfolio valuation

Do NOT hard-code the architecture around a tiny list of stocks.

Create a stock data structure that can eventually support the broader Philippine stock universe.

However:

Do not assume that a particular external market-data provider exists.

Before implementing production market-data integration, investigate the available data source/API and its:

- availability
- API structure
- rate limits
- licensing
- reliability
- cost
- allowed usage

If an external provider is required, ask me before committing the project to a paid or restrictive service.

11. NEWS DATA

The News tab should eventually use real Philippine finance/business news sources.

The architecture should separate:

NEWS UI

from

NEWS DATA SOURCE.

Do not hard-code news directly into UI components.

Create an abstraction that allows the news source to be changed later.

News items should conceptually contain:

- title
- source
- URL
- published date
- image URL if available
- summary/excerpt
- relevant stock/company information when available

Avoid copying entire articles.

The application should primarily display a useful preview and allow the user to open the original source.

12. PHASED DEVELOPMENT

DO NOT BUILD EVERYTHING AT ONCE.

We will build this application one phase at a time.

The phases are:

PHASE 1 — FRONT-END / UI

PHASE 2 — CORE PORTFOLIO LOOP

PHASE 3 — ACCESSORY FEATURES / DATA

PHASE 4 — SURFACE AREA / UX REFINEMENT

PHASE 5 — RETENTION

FINAL PHASE — SHIPPABLE MVP

PHASE 1 — FRONT-END

Phase 1 is FRONT-END FIRST.

I will provide reference images for the application's UI.

Use those references to build the front-end.

The objective is to produce a convincing, functional UI prototype before building the full backend.

Build the main application surfaces:

- Portfolio
- Stocks
- News
- Settings
- Authentication screens where needed for the visual flow

Use realistic mock data.

The Portfolio screen should visually demonstrate:

- total portfolio value
- performance/growth

- graph
- holdings
- stock values
- add holding action

The Stocks screen should visually demonstrate:

- searchable stock list
- Philippine stock symbols
- company names
- prices
- changes

The News screen should visually demonstrate:

- news cards
- headline
- source
- date
- relevant stock/business news

Settings should visually demonstrate:

- account
- preferences
- logout

At this phase, the primary objective is:

LOOK + FEEL + BASIC INTERACTION

Do not spend Phase 1 building complex backend systems.

13. REFERENCE IMAGE WORKFLOW

When I give you a reference image for a screen:

FIRST:

Analyze the reference.

Identify:

- layout

- hierarchy
- navigation
- typography
- spacing
- cards
- buttons
- icons
- colors
- graphs
- lists
- empty states
- interactions

SECOND:

Translate the reference into reusable React Native components.

THIRD:

Implement it in the existing application.

FOURTH:

Run it locally.

FIFTH:

Open it in Chrome where supported.

SIXTH:

Compare the actual implementation against the reference.

SEVENTH:

Fix obvious differences and issues.

Do not simply create a static image.

The result must be a real working interface.

14. LOCALHOST + CHROME REQUIREMENT

After Phase 1 implementation:

Run the application using the existing development workflow.

Make the frontend available through localhost.

When browser testing is available:

open the localhost application in a new Chrome window/tab and test it.

Actually interact with it.

Test:

- navigation
- buttons
- tabs
- add holding interaction
- stock list
- news list
- settings
- authentication UI
- responsive behavior
- obvious runtime errors

Do not say:

"It should work."

Verify it.

If something does not work:

Fix it before declaring the phase complete.

PHASE 2 — CORE PORTFOLIO LOOP

After I explicitly approve Phase 1:

Turn the frontend prototype into a functional portfolio tracker.

Implement:

- authentication
- user session
- user-specific portfolio
- holdings
- add holding
- increase holding
- decrease holding
- remove holding
- portfolio calculation
- individual holding value
- total portfolio value
- portfolio performance data
- portfolio graph data

The core loop must work end-to-end.

The UI from Phase 1 should be preserved unless a change is necessary.

Do not redesign the entire application during Phase 2.

PHASE 3 — ACCESSORY FEATURES / REAL DATA

After I approve Phase 2:

Implement the necessary supporting features.

Potentially:

- Philippine stock database
- real market data
- stock search
- stock detail information
- Philippine financial news
- news source integration
- historical price data
- better portfolio calculations
- relevant loading/error states

Only implement features that support the core product.

Do not add random finance features.

PHASE 4 — SURFACE AREA CHECK

Audit the entire application.

Ask:

"Can this application be simpler?"

Look for:

- unnecessary screens
- unnecessary navigation
- redundant actions
- unnecessary settings
- overly complicated portfolio management
- confusing stock flows
- excessive information
- redundant UI

The target is approximately 4–5 primary surfaces unless more are clearly necessary.

Make the primary workflow extremely obvious:

Open → Understand portfolio → Update holdings → See result.

PHASE 5 — RETENTION HOOK

After the core product is stable, determine the most natural reason users would return.

Possible mechanisms:

- portfolio history
- performance tracking
- personalized stock/news updates
- watchlist
- meaningful market updates

- progress/history
- notifications

Do not add artificial gamification just to create engagement.

The retention hook should reinforce the application's core value.

If the correct retention strategy is unclear, stop and ask me before implementing a major feature.

FINAL PHASE — SHIPPABLE MVP

Only after all phases are approved:

Perform a complete audit.

Verify:

AUTHENTICATION

- Sign up works
- Login works
- Logout works
- Sessions work
- Users cannot access other users' private data

PORTFOLIO

- Holdings can be added
- Holdings can be increased
- Holdings can be decreased
- Holdings can be removed
- Total portfolio value is calculated correctly
- Individual holding values are calculated correctly

PERFORMANCE

- Portfolio history works
- Graph works
- Performance calculations are consistent with available data

STOCKS

- Philippine stock list works

- Search works
- Stock information loads correctly

NEWS

- News loads correctly
- Sources are displayed
- Original articles can be opened
- No full copyrighted articles are reproduced

UI / UX

- Navigation works
- No dead ends
- No broken buttons
- Loading states exist where needed
- Error states exist where needed
- Empty states exist where needed
- Mobile layout is usable
- Web/Chrome testing works where supported

TECHNICAL QUALITY

Check for:

- TypeScript errors
- runtime errors
- broken imports
- unnecessary dependencies
- duplicated components
- obvious performance problems
- security issues
- exposed secrets
- insecure data access

15. SECURITY RULES

This is a financial application, so security and data isolation are important.

Never:

- expose API secrets in frontend code
- expose private environment variables
- trust user IDs supplied by the client for authorization
- allow users to query another user's portfolio
- rely only on frontend filtering for access control

Always enforce authorization at the appropriate backend/database layer.

If a security decision materially affects the architecture, stop and ask me before proceeding.

16. SCOPE RULE

Do not build:

- stock trading execution
- brokerage integration
- bank account integration
- cryptocurrency
- international stocks
- options
- futures
- advanced technical analysis
- social trading
- complicated financial planning
- unnecessary gamification

unless I explicitly request them.

The MVP should stay focused.

17. WHEN TO ASK ME

Do not ask me about trivial implementation decisions.

Make reasonable engineering decisions when the intent is clear.

Ask me when a decision materially affects:

- architecture

- authentication
 - database
 - security
 - core user flow
 - external API provider
 - paid service
 - major product feature
 - data licensing
 - irreversible migration
-

18. PHASE GATE — MANDATORY

At the end of EVERY phase, STOP.

Do not automatically start the next phase.

Report:

PHASE [NUMBER] COMPLETE

What was implemented

- ...

Important files changed

- ...

Localhost

- URL: ...
- Status: ...

What I tested

- ...
- ...
- ...

Known issues

- ...

Recommended next phase

- ...

Then WAIT.

Only continue when I explicitly say something like:

"Continue to Phase 2."

or

"Approved. Move to the next phase."

19. MOST IMPORTANT RULE

DO NOT BUILD THE ENTIRE APP IN ONE PASS.

DO NOT SKIP PHASES.

DO NOT MOVE TO THE NEXT PHASE WITHOUT MY EXPLICIT APPROVAL.

The development sequence is:

REFERENCE

↓

PHASE 1 — FRONT-END

↓

LOCALHOST

↓

CHROME TEST

↓

MY APPROVAL

↓

PHASE 2 — CORE PORTFOLIO LOOP

↓

TEST

↓

MY APPROVAL

↓

PHASE 3 — REAL DATA + ACCESSORY FEATURES

↓

TEST



MY APPROVAL



PHASE 4 — SURFACE AREA / UX



TEST



MY APPROVAL



PHASE 5 — RETENTION



TEST



MY APPROVAL



FINAL — SHIPPABLE MVP

The guiding principle is:

**SIMPLE → FUNCTIONAL → TESTED → APPROVED → NEXT
PHASE**

Build a focused Philippine stock portfolio application.

Do not build a complicated trading terminal.

The goal is for a user to open the app and immediately understand:

**"How much is my portfolio worth, what do I own, how is it
performing, and what's happening in the Philippine stock
market?"**

Voice chat ended

5s